MOMO Quant

Part 8 — Cursor Implementation Plan

Version: 1.0 Draft
Purpose: Convert the project documents into a step-by-step Cursor build plan.
Primary Rule: Cursor must build MOMO Quant module by module, not as one giant generation.

---

1. Implementation Philosophy

MOMO Quant must not be generated in one massive prompt.

That would produce:

- inconsistent architecture
- duplicated logic
- weak error handling
- poor separation of concerns
- broken tests
- missing abstractions
- dangerous trading shortcuts

The correct approach is:

```
Document first
Skeleton second
Infrastructure third
Core modules fourth
Trading logic fifth
UI sixth
Live trading last
```

Cursor should behave like a developer following a professional specification.

---

2. Source of Truth Documents

Cursor must treat these documents as the project authority:

```
Part 1 — Software Requirements Specification
Part 2 — System Architecture Document
Part 3 — Database Design Document
Part 4 — API Specification
Part 5 — AI & Machine Learning Design Document
Part 6 — UI/UX Specification
Part 7 — DevOps & Deployment Guide
Part 8 — Cursor Implementation Plan
```

These should be saved in the repository:

```
/docs/01-srs.md
/docs/02-system-architecture.md
/docs/03-database-design.md
/docs/04-api-specification.md
/docs/05-ai-ml-design.md
/docs/06-ui-ux-specification.md
/docs/07-devops-deployment.md
/docs/08-cursor-implementation-plan.md
```

Cursor should be told to read these before implementing modules.

---

3. Cursor Global Rules

Every Cursor prompt should include these rules:

```
Follow the MOMO Quant documentation in /docs.

Do not invent architecture that conflicts with the documents.

Use .NET 8 for backend.

Use React + Vite + Tailwind CSS for frontend.

Use MySQL with Entity Framework Core.

Use Redis where runtime state/cache is needed.

Use Python FastAPI for the AI service.

Build as a modular monolith first.
```

```
Do not create microservices except the Python AI service.

Do not implement live trading in MVP.

Do not let AI place orders.

Do not let strategies place orders.

Risk engine must approve every trade before execution.

Backtesting, replay, paper trading, and future live trading must share common
abstractions.

Every important trading decision must be logged.

All timestamps must use UTC.

All trading financial values must use decimal, not float/double.

Use clean, readable code.

Add tests where possible.

When modifying existing files, return the full updated file.
```

---

4. Recommended Repository Structure

Cursor should create this structure:

```
momo-quant/
  docs/

  src/
    backend/
      MomoQuant.sln
      src/
        MomoQuant.Api/
        MomoQuant.Application/
        MomoQuant.Domain/
        MomoQuant.Infrastructure/
        MomoQuant.Persistence/
        MomoQuant.Worker/
        MomoQuant.Shared/
      tests/
```

```
      MomoQuant.UnitTests/
      MomoQuant.IntegrationTests/

  frontend/
    momo-dashboard/

  ai/
    momo-ai/

deploy/
  docker-compose.yml
  docker-compose.override.yml
  .env.example

scripts/
  backup-db.sh
  restore-db.sh
  deploy.sh
  logs.sh

README.md
```

---

5. Implementation Milestones

Build MOMO Quant in these milestones:

```
Milestone 1  — Repository and solution skeleton
Milestone 2  — Backend domain models and shared contracts
Milestone 3  — Database entities and EF Core setup
Milestone 4  — Authentication and authorization
Milestone 5  — Exchange and symbol management
Milestone 6  — Historical candle import and storage
Milestone 7  — Indicator engine
Milestone 8  — Strategy framework
Milestone 9  — Risk engine
Milestone 10 — AI service skeleton
Milestone 11 — .NET AI integration
Milestone 12 — Backtesting engine
Milestone 13 — Replay engine
Milestone 14 — Paper trading engine
Milestone 15 — Reporting engine
Milestone 16 — Monitoring and audit logging
Milestone 17 — React dashboard shell
Milestone 18 — Dashboard feature pages
```

```
Milestone 19 — Docker local environment
Milestone 20 — Deployment preparation
Milestone 21 — Live trading readiness only
```

Live execution should not be implemented until backtesting and paper trading are proven stable.

---

6. Milestone 1 — Repository and Solution Skeleton

Goal

Create the initial repository structure and .NET solution.

Cursor Prompt

```
You are building MOMO Quant, an AI-assisted quantitative crypto futures trading
platform.

Read the documentation in /docs before coding.

Create the initial repository structure for:

- .NET 8 backend
- React + Vite frontend
- Python FastAPI AI service
- Docker deployment files
- scripts folder
- docs folder

Inside src/backend, create a .NET 8 solution named MomoQuant.sln with these
projects:

- MomoQuant.Api
- MomoQuant.Application
- MomoQuant.Domain
- MomoQuant.Infrastructure
- MomoQuant.Persistence
- MomoQuant.Worker
- MomoQuant.Shared
- MomoQuant.UnitTests
- MomoQuant.IntegrationTests

Set correct project references:
```

```
Api references Application, Infrastructure, Persistence, Shared.
Application references Domain and Shared.
Infrastructure references Application, Domain, Shared.
Persistence references Domain, Application, Shared.
Worker references Application, Infrastructure, Persistence, Shared.
Tests reference the necessary projects.

Do not implement trading logic yet.
Only create the skeleton, references, basic Program.cs, and README instructions.

Return the full created/updated files.
```

Acceptance Criteria

```
Solution builds.
All projects exist.
Project references are correct.
No trading logic is implemented yet.
Repository structure matches the documentation.
```

---

7. Milestone 2 — Domain Models and Shared Contracts

Goal

Create core domain models without database dependencies.

Cursor Prompt

```
Implement the core domain models for MOMO Quant according to /docs.

Create domain models and enums for:

- Exchange
- Symbol
- Candle
- IndicatorSnapshot
- Strategy
- StrategyParameter
- TradingSession
- StrategySignal
```

```
- AiDecision
- RiskDecision
- Order
- OrderFill
- MissedOrder
- Trade
- Position
- BacktestRun
- BacktestResult
- ReplaySession
- PaperAccount
- PaperAccountSnapshot
- AuditLog
- SystemHealthLog
- AppSetting

Create enums for:

- TradingMode
- TradingSessionStatus
- Timeframe
- MarketRegime
- SignalType
- TradeDirection
- OrderSide
- OrderType
- OrderStatus
- PositionStatus
- RiskDecisionType
- TradeStatus
- CloseReason
- LiquidityType
- HealthStatus

Rules:

- Use decimal for all financial values.
- Use DateTime UTC fields for timestamps.
- Do not add EF Core attributes yet unless necessary.
- Keep domain clean and independent.
- Do not implement database logic in Domain.
- Add basic guard methods where useful.

Return full files.
```

Acceptance Criteria

```
Domain project builds.
No dependency on EF Core in Domain.
Financial values use decimal.
Enums are clear and match documentation.
```

---

8. Milestone 3 — Database Entities and EF Core Setup

Goal

Create persistence layer and migrations.

Cursor Prompt

```
Implement MOMO Quant persistence layer using Entity Framework Core and MySQL.

Use Pomelo.EntityFrameworkCore.MySql.

Create MomoQuantDbContext.

Map the domain entities from /docs/03-database-design.md.

Create DbSet properties for MVP tables:

- Users
- Roles
- Exchanges
- Symbols
- Candles
- Strategies
- StrategyParameters
- TradingSessions
- TradingSessionSymbols
- IndicatorSnapshots
- StrategySignals
- AiDecisions
- RiskProfiles
- RiskRules
- RiskDecisions
```

```
- Orders
- OrderFills
- MissedOrders
- Trades
- Positions
- BacktestRuns
- BacktestResults
- ReplaySessions
- PaperAccounts
- PaperAccountSnapshots
- AuditLogs
- SystemHealthLogs
- AppSettings

Configure:

- decimal precision decimal(28,12)
- UTC timestamp conventions
- required indexes from database design document
- unique candle index on ExchangeId + SymbolId + Timeframe + OpenTimeUtc
- relationships
- delete restrictions for trading records

Add design-time DbContext factory.

Add initial migration.

Do not create live trading logic.

Return full files.
```

Acceptance Criteria

```
Database project builds.
Migration can be created.
DbContext contains MVP tables.
Indexes are configured.
Decimal precision is configured.
No plain text API secrets are exposed.
```

---

9. Milestone 4 — Authentication and Authorization

Goal

Implement JWT login and roles.

Cursor Prompt

```
Implement authentication and authorization for MOMO Quant.

Follow /docs/04-api-specification.md.

Create:

- AuthController
- UsersController
- JWT authentication
- role-based authorization
- password hashing service
- current user service
- seed data for roles: Admin, Trader, Viewer
- optional seed admin user for development only

APIs:

POST /api/v1/auth/login
GET /api/v1/auth/me
POST /api/v1/auth/logout

Admin user APIs:

GET /api/v1/users
GET /api/v1/users/{id}
POST /api/v1/users
PUT /api/v1/users/{id}
POST /api/v1/users/{id}/disable

Use standard response format from API spec.

Do not implement trading modules yet.

Return full files.
```

Acceptance Criteria

```
Login works.
JWT is returned.
Role authorization works.
Admin-only APIs are protected.
Standard response format is used.
```

---

10. Milestone 5 — Exchange and Symbol Management

Goal

Add exchanges and symbols before market data.

Cursor Prompt

```
Implement Exchange and Symbol management APIs for MOMO Quant.

Follow /docs/04-api-specification.md.

Create:

- Exchange service
- Symbol service
- ExchangeController
- SymbolsController
- DTOs
- validators
- audit logging for state-changing operations

APIs:

GET /api/v1/exchanges
POST /api/v1/exchanges
PUT /api/v1/exchanges/{id}
POST /api/v1/exchanges/{id}/test-connection

GET /api/v1/symbols
POST /api/v1/symbols/sync
PUT /api/v1/symbols/{id}/status
```

```
For symbol sync, create a provider interface:

IExchangeSymbolProvider

Implement a fake/local provider first.

Do not connect to real exchange yet unless clearly isolated behind the
interface.

Return full files.
```

Acceptance Criteria

```
Exchange CRUD works.
Symbols can be listed.
Fake symbol sync works.
Top 5 symbols can be seeded.
Exchange integration is abstracted.
```

---

11. Milestone 6 — Historical Candle Import and Storage

Goal

Store historical candle data.

Cursor Prompt

```
Implement historical candle import and candle storage for MOMO Quant.

Follow /docs/03-database-design.md and /docs/04-api-specification.md.

Create:

- Candle entity mapping if missing
- MarketDataController
- Candle import service
- Candle query service
- Candle repository/query methods
- Historical data provider interface
```

```
APIs:

GET /api/v1/market-data/candles
POST /api/v1/market-data/candles/import
GET /api/v1/market-data/imports/{importId}
GET /api/v1/market-data/snapshot

For MVP, support importing candles from a local JSON/CSV file or fake provider.
Real exchange historical API can be added later behind the provider interface.

Rules:

- Use UTC timestamps.
- Enforce unique candle index.
- Batch insert candles.
- Do not duplicate existing candles.
- Return import status.

Return full files.
```

Acceptance Criteria

```
Candles can be imported.
Candles can be queried by symbol/timeframe/date range.
Duplicates are prevented.
Data uses UTC.
```

---

12. Milestone 7 — Indicator Engine

Goal

Calculate indicators outside strategies.

Cursor Prompt

```
Implement the Indicator Engine for MOMO Quant.

Follow /docs/02-system-architecture.md and /docs/03-database-design.md.

Create indicator calculation services for:
```

```
- EMA20
- EMA50
- EMA200
- VWAP
- RSI14
- ATR14
- VolumeSMA20
- SwingHigh
- SwingLow
- Basic market structure

Create:

- IIndicatorCalculator
- IndicatorCalculationService
- IndicatorController

APIs:

GET /api/v1/indicators/snapshot
POST /api/v1/indicators/recalculate

Rules:

- Indicators must be deterministic.
- Strategies must not calculate indicators directly.
- Store results in IndicatorSnapshots.
- Add unit tests for EMA, RSI, ATR basics.

Return full files.
```

Acceptance Criteria

```
Indicators calculate correctly enough for MVP.
Indicator snapshots are stored.
Strategies can consume indicator snapshots.
Unit tests exist for core calculations.
```

---

13. Milestone 8 — Strategy Framework

Goal

Create strategy plugin architecture and first strategies.

Cursor Prompt

```
Implement the MOMO Quant strategy framework.

Follow /docs/02-system-architecture.md, /docs/05-ai-ml-design.md, and /docs/06-
ui-ux-specification.md.

Create:

- ITradingStrategy
- StrategyContext
- StrategySignalResult
- StrategyEngine
- Strategy registry
- Strategy parameter provider

Implement MVP strategies:

1. LiquiditySweepStrategy
2. VwapMeanReversionStrategy
3. EmaPullbackStrategy

Rules:

- Strategies only produce signals.
- Strategies must not place orders.
- Strategies must not approve risk.
- Each strategy must return human-readable reason.
- Each strategy must declare supported regimes and timeframes.
- Store generated signals in StrategySignals.

Also implement StrategyController APIs:

GET /api/v1/strategies
GET /api/v1/strategies/{id}
POST /api/v1/strategies/{id}/enable
POST /api/v1/strategies/{id}/disable
GET /api/v1/strategies/{id}/parameters
PUT /api/v1/strategies/{id}/parameters

Return full files.
```

Acceptance Criteria

```
Strategy engine runs all enabled strategies.
Each strategy returns signal/no-trade result.
Signals are stored.
Strategies do not execute trades.
Strategy parameters can be changed.
```

---

14. Milestone 9 — Risk Engine

Goal

Implement final gate before execution.

Cursor Prompt

```
Implement the MOMO Quant Risk Engine.

Follow /docs/03-database-design.md and /docs/05-ai-ml-design.md.

Create:

- IRiskEngine
- RiskContext
- RiskEvaluationResult
- RiskProfileService
- RiskRuleService
- RiskDecision logging

Risk rules:

- MaxRiskPerTradePercent
- MaxDailyLossPercent
- MaxWeeklyLossPercent
- MaxOpenPositions
- MaxExposurePerSymbol
- MaxCorrelatedExposure
- MaxConsecutiveLosses
- MinConfidenceScore
- MaxSpreadPercent
- MaxVolatility
- EmergencyStopEnabled
```

```
APIs:

GET /api/v1/risk/profiles
POST /api/v1/risk/profiles
GET /api/v1/risk/profiles/{id}/rules
PUT /api/v1/risk/profiles/{id}/rules
GET /api/v1/risk/decisions

Rules:

- Risk engine has final authority.
- Risk engine can reject AI-approved trades.
- Risk decisions must be stored.
- Risk changes must create audit logs.

Return full files.
```

Acceptance Criteria

```
Risk profile and rules work.
Risk engine approves/rejects trades.
Rejection reasons are clear.
Risk decisions are stored.
Emergency stop blocks trades.
```

---

15. Milestone 10 — Python AI Service Skeleton

Goal

Create FastAPI AI service.

Cursor Prompt

```
Create the Python FastAPI AI service for MOMO Quant.

Follow /docs/05-ai-ml-design.md.

Create structure:

momo-ai/
  app/
```

```
    main.py
    api/
    core/
    services/
    models/
    features/
    storage/
    tests/

Implement endpoints:

GET /health
POST /ai/regime/detect
POST /ai/confidence/score
POST /ai/anomaly/detect
POST /ai/trade/explain
POST /ai/parameters/optimize

Use:

- FastAPI
- Pydantic
- pandas
- numpy
- scikit-learn optional later
- pytest

For MVP, implement rule-based logic:

- market regime detection
- weighted confidence scoring
- simple anomaly detection
- explanation generation

Do not implement deep learning.
Do not implement autonomous trading.
Do not call exchange APIs.

Return full files.
```

Acceptance Criteria

```
AI service runs locally.
Health endpoint works.
All AI endpoints return structured JSON.
```

```
Rule-based AI logic is explainable.
Tests exist for basic AI behavior.
```

---

16. Milestone 11 — .NET AI Integration

Goal

Connect backend to Python AI service.

Cursor Prompt

```
Integrate the .NET backend with the Python AI service.

Create:

- IAiServiceClient
- AiServiceClient
- request/response DTOs
- timeout handling
- retry policy if appropriate
- fallback handling
- AiDecision persistence
- AI dashboard-facing APIs

APIs:

GET /api/v1/ai/decisions
GET /api/v1/ai/decisions/{id}
POST /api/v1/ai/regime/test
GET /api/v1/ai/performance

Rules:

- AI output must be stored in AiDecisions.
- AI cannot place orders.
- AI failures must be handled safely.
- In paper/live mode, failed AI should block or fallback depending on config.
- In backtest/replay, fallback is allowed.

Return full files.
```

Acceptance Criteria

```
.NET can call Python AI service.
AI decisions are stored.
AI errors do not crash the app.
AI APIs return stored decision data.
```

---

17. Milestone 12 — Backtesting Engine

Goal

Run historical strategy tests.

Cursor Prompt

```
Implement the MOMO Quant Backtesting Engine.

Follow all docs, especially:

- /docs/01-srs.md
- /docs/02-system-architecture.md
- /docs/03-database-design.md
- /docs/04-api-specification.md

Create:

- IBacktestEngine
- BacktestRunner
- BacktestExecutionProvider
- BacktestPortfolio
- BacktestPerformanceCalculator
- BacktestController

API:

POST /api/v1/backtests/run
GET /api/v1/backtests/{id}
GET /api/v1/backtests/{id}/results
GET /api/v1/backtests/{id}/trades
GET /api/v1/backtests/{id}/signals
POST /api/v1/backtests/{id}/cancel
```

```
Backtest flow:

- load candles
- calculate/load indicators
- detect market regime
- run strategy engine
- call AI confidence engine
- run risk engine
- simulate execution
- store signals, AI decisions, risk decisions, orders, fills, trades
- calculate results

Execution simulation must include:

- maker fee
- taker fee
- slippage assumption
- order timeout
- missed maker orders

Return full files.
```

Acceptance Criteria

```
Backtest can run from API.
Backtest stores decisions and trades.
Backtest results are calculated.
Missed maker orders are logged.
Results are realistic enough for MVP.
```

---

18. Milestone 13 — Replay Engine

Goal

Replay historical decisions for debugging.

Cursor Prompt

```
Implement Replay Mode for MOMO Quant.

Replay mode should reuse backtesting logic but expose step-by-step timeline
state.
```

```
Create:

- ReplayEngine
- ReplaySessionService
- ReplayState
- ReplayController
- ReplayHub using SignalR

APIs:

POST /api/v1/replay/create
GET /api/v1/replay/{id}
POST /api/v1/replay/{id}/start
POST /api/v1/replay/{id}/pause
POST /api/v1/replay/{id}/resume
POST /api/v1/replay/{id}/speed
POST /api/v1/replay/{id}/step
POST /api/v1/replay/{id}/stop

SignalR events:

- ReplayStarted
- ReplayPaused
- ReplayStopped
- ReplayTick
- ReplayDecisionUpdated
- ReplayTradeUpdated

Replay tick must include:

- current candle
- indicators
- strategy signal
- AI decision
- risk decision
- execution decision
- trade updates
- no-trade reason

Return full files.
```

Acceptance Criteria

```
Replay session can be created.
Replay can step candle-by-candle.
```

```
Replay emits SignalR events.
Timeline shows trade/no-trade decisions.
Replay reuses core trading logic.
```

---

19. Milestone 14 — Paper Trading Engine

Goal

Use live market data with simulated execution.

Cursor Prompt

```
Implement Paper Trading for MOMO Quant.

Create:

- PaperTradingEngine
- PaperExecutionProvider
- PaperPortfolioService
- PaperAccountService
- PaperTradingController
- TradingHub events

APIs:

GET /api/v1/paper/accounts
POST /api/v1/paper/accounts
POST /api/v1/paper/start
POST /api/v1/paper/stop
GET /api/v1/paper/accounts/{id}/snapshot
GET /api/v1/paper/accounts/{id}/trades

Rules:

- Paper trading uses same strategy, AI, risk, and execution abstractions as live
trading would.
- Only execution provider is simulated.
- Simulate limit orders, partial fills if possible, fees, and missed maker
orders.
- Store orders, fills, trades, positions, decisions.
- Show PAPER MODE clearly in status responses.
- Do not implement real exchange order placement.
```

```
Return full files.
```

Acceptance Criteria

```
Paper account can be created.
Paper trading can start/stop.
Paper execution stores simulated orders and trades.
Paper trading uses same core decision flow.
No real orders are placed.
```

---

20. Milestone 15 — Reporting Engine

Goal

Turn stored data into useful reports.

Cursor Prompt

```
Implement the MOMO Quant Reporting Engine.

Create:

- ReportingController
- PerformanceReportService
- StrategyReportService
- SymbolReportService
- MarketRegimeReportService
- RejectionReportService
- MissedTradeReportService
- AiPerformanceReportService

APIs:

GET /api/v1/reports/dashboard-summary
GET /api/v1/reports/performance
GET /api/v1/reports/strategies
GET /api/v1/reports/symbols
GET /api/v1/reports/market-regimes
GET /api/v1/reports/rejections
GET /api/v1/reports/missed-trades
```

```
Metrics:

- total trades
- win rate
- net PnL
- gross PnL
- fees
- profit factor
- expectancy
- max drawdown
- average win
- average loss
- performance by strategy
- performance by symbol
- performance by regime
- rejected reason counts
- missed trade counts

Return full files.
```

Acceptance Criteria

```
Reports are generated from stored facts.
Performance metrics are calculated.
Rejected and missed trade reports work.
Dashboard summary works.
```

---

21. Milestone 16 — Monitoring and Audit Logging

Goal

Make system behavior visible and traceable.

Cursor Prompt

```
Implement monitoring and audit logging for MOMO Quant.

Create:

- AuditLogService
- MonitoringService
- HealthCheckService
```

```
- MonitoringController
- AuditLogsController
- MonitoringHub

APIs:

GET /api/v1/monitoring/health
GET /api/v1/monitoring/exchange-health
GET /api/v1/monitoring/workers
GET /api/v1/monitoring/errors
GET /api/v1/audit-logs
GET /api/v1/audit-logs/{id}

Audit these actions:

- login
- user changes
- exchange changes
- API key changes
- strategy parameter changes
- risk rule changes
- bot start/stop
- mode changes
- emergency stop
- backtest start/cancel
- replay start/stop
- paper trading start/stop

Return full files.
```

Acceptance Criteria

```
Health endpoint works.
Audit logs are created for state changes.
Monitoring data is available to dashboard.
Critical system states are visible.
```

---

22. Milestone 17 — React Dashboard Shell

Goal

Create frontend foundation.

Cursor Prompt

```
Create the React + Vite + Tailwind CSS dashboard shell for MOMO Quant.

Follow /docs/06-ui-ux-specification.md.

Create:

- app router
- layout shell
- top bar
- sidebar navigation
- auth provider
- API client
- SignalR client setup
- protected routes
- role-based navigation
- reusable components

Reusable components:

- ModeBadge
- BotStatusBadge
- HealthBadge
- EmergencyStopButton
- MetricCard
- DataTable
- ConfirmationModal
- PageHeader
- EmptyState
- ErrorState
- LoadingState

Pages as placeholders first:

- Login
- Dashboard
- Bot Control
- Market Watch
- Strategies
- Backtesting
- Replay Mode
- Paper Trading
- Trades
- Orders
- Positions
```

```
- Reports
- AI Analytics
- Risk Management
- Monitoring
- Logs
- Settings
- Admin

Do not implement complex UI logic yet.
Build the shell cleanly.

Return full files.
```

Acceptance Criteria

```
Frontend runs locally.
Login page exists.
Dashboard shell exists.
Sidebar works.
Protected routes work.
Reusable UI components exist.
```

---

23. Milestone 18 — Dashboard Feature Pages

Goal

Connect UI to APIs.

Cursor Prompt

```
Implement MOMO Quant dashboard feature pages using existing APIs.

Follow /docs/06-ui-ux-specification.md and /docs/04-api-specification.md.

Implement pages:

- Dashboard home
- Bot Control
- Exchange Settings
- Symbol Settings
- Strategy list/details
- Risk Management
```

```
- Market Watch
- Backtesting form/results
- Replay Mode
- Paper Trading
- Trades
- Orders
- Positions
- Reports
- AI Analytics
- Monitoring
- Logs
- Audit Logs

Rules:

- Use React Query for API state.
- Use SignalR hooks for realtime events.
- Use clear loading/error/empty states.
- Dangerous actions require confirmation.
- Live trading UI must remain locked.
- Paper mode must be visibly marked.
- Emergency stop must always be accessible.

Return full files.
```

Acceptance Criteria

```
Dashboard connects to backend.
Backtests can be started from UI.
Replay can be controlled from UI.
Paper trading can be started/stopped from UI.
Reports display real data.
Monitoring updates are visible.
```

---

24. Milestone 19 — Docker Local Environment

Goal

Run dependencies locally.

Cursor Prompt

```
Create Docker local development setup for MOMO Quant.

Follow /docs/07-devops-deployment.md.

Create:

- docker-compose.yml
- docker-compose.override.yml
- .env.example
- MySQL service
- Redis service
- optional API service
- optional AI service
- optional dashboard service

For local development, support this workflow:

- MySQL and Redis in Docker
- .NET API running locally
- React dashboard running locally
- Python AI service running locally

Also support full Docker Compose later.

Add README instructions for:

- starting MySQL/Redis
- running migrations
- running backend
- running frontend
- running AI service
- stopping services
- resetting local database

Return full files.
```

Acceptance Criteria

```
MySQL runs locally in Docker.
Redis runs locally in Docker.
.env.example exists.
```

```
Local setup instructions are clear.
No real secrets are committed.
```

---

25. Milestone 20 — Deployment Preparation

Goal

Prepare for VPS deployment later.

Cursor Prompt

```
Prepare MOMO Quant for Docker Compose deployment.

Follow /docs/07-devops-deployment.md.

Create:

- production Dockerfile for .NET API
- production Dockerfile for React dashboard
- production Dockerfile for Python AI service
- Nginx config
- docker-compose production profile
- backup-db.sh
- restore-db.sh
- deploy.sh
- logs.sh
- health checks
- production README deployment guide

Rules:

- Only Nginx exposes public ports.
- MySQL, Redis, API internal ports, and AI service must not be public.
- Use environment variables.
- Do not commit secrets.
- Include HTTPS notes.
- Include backup-before-migration instruction.

Return full files.
```

Acceptance Criteria

```
Production containers build.
Nginx routes dashboard/API/SignalR.
Backup script exists.
Restore script exists.
Deployment instructions are clear.
```

---

26. Milestone 21 — Live Trading Readiness Only

Goal

Prepare safety checks without real trading.

Cursor Prompt

```
Implement Live Trading Readiness checks for MOMO Quant.

Do not implement real live order execution yet.

Create:

- LiveReadinessService
- LiveController

APIs:

GET /api/v1/live/readiness
POST /api/v1/live/enable
POST /api/v1/live/disable

Rules:

- Live enable must remain blocked unless all readiness checks pass.
- Readiness checks include:
  - exchange API key exists
  - API key test passes
  - risk profile active
  - emergency stop available
  - paper trading validation completed
  - system health is healthy
  - AI service healthy or fallback configured
```

```
  - admin confirmation text provided
- Enabling live mode must be audited.
- Real exchange order placement must not be implemented in this milestone.

Return full files.
```

Acceptance Criteria

```
Live readiness endpoint works.
Live mode remains safely locked.
Admin confirmation is required.
Audit logs are created.
No real orders can be placed.
```

---

27. Cursor Prompt Pattern

Every future prompt should follow this structure:

```
Context:
You are working on MOMO Quant. Read the relevant docs in /docs.

Task:
Implement [specific module].

Requirements:
List exact requirements.

Files:
Mention files/modules to modify or create.

Rules:
Mention safety and architecture rules.

Acceptance Criteria:
List what must work.

Output:
Return full updated files.
```

Do not use vague prompts like:

```
Build the trading bot.
```

That is how bad code gets created.

---

28. Development Branch Strategy

Use simple Git branches:

```
main
develop
feature/milestone-01-skeleton
feature/milestone-02-domain
feature/milestone-03-persistence
```

Rules:

```
main should always be stable.
develop is integration branch.
Each milestone gets a feature branch.
Commit after every working module.
Do not mix unrelated modules in one branch.
```

---

29. Definition of Done

A module is not done just because it compiles.

A milestone is done only when:

```
Code builds.
Tests pass.
Database migration works if applicable.
API endpoint works if applicable.
UI page works if applicable.
Errors are handled.
Audit logs are created for state changes.
No secrets are committed.
```

```
Documentation updated if needed.
Manual smoke test completed.
```

---

30. Testing Requirements

Cursor should add tests gradually.

Initial tests:

```
Domain unit tests
Indicator calculation tests
Strategy signal tests
Risk engine tests
AI service tests
Backtest engine tests
API integration tests later
```

High-priority test targets:

```
Risk engine
Backtesting engine
Execution simulation
Indicator calculations
Strategy decisions
AI scoring
Mode safety
Emergency stop
```

Do not skip risk engine tests.

That is the one area where bugs can become expensive later.

---

31. Local Build Order

Before deployment, run locally in this order:

```
1. MySQL + Redis
2. Backend migrations
3. Backend API
```

```
4. Python AI service
5. React dashboard
6. Candle import
7. Indicator recalculation
8. Backtest
9. Replay
10. Paper trading
11. Reports
12. Monitoring
```

Do not deploy until this works locally.

---

32. First End-to-End Local Test

The first meaningful test should be:

```
Login as admin
Create exchange
Sync or seed symbols
Import BTCUSDT candles
Recalculate indicators
Enable EMA Pullback strategy
Create conservative risk profile
Run backtest
View signals
View AI decisions
View risk decisions
View trades
View report
Open replay session
Step through decisions
```

If this flow works, the platform foundation is real.

---

33. What Not to Build Early

Do not build these early:

```
Real live trading execution
Kubernetes
Multi-exchange routing
```

```
Advanced AI optimization
Reinforcement learning
Mobile app
Subscription billing
Public SaaS features
Advanced notification integrations
Order book strategy logic
News sentiment engine
```

These are distractions before the core system is validated.

---

34. Biggest Engineering Risks

Main risks:

```
Scope creep
Unrealistic backtests
Ignoring fees/slippage
Maker orders not filling
Weak logging
AI overfitting
Bad risk rules
Messy frontend state
Too many services too early
Moving to live trading too early
```

Cursor cannot solve these automatically.

The architecture must force discipline.

---

35. Final Cursor Implementation Summary

Cursor should be used as a disciplined coding assistant, not as a magic product generator.

The correct instruction is never:

```
Build the whole app.
```

The correct instruction is:

```
Build this specific module according to these documents and acceptance criteria.
```

The most important implementation rule:

```
Do not optimize for speed of generation.
Optimize for code that can be understood, tested, debugged, and safely extended.
```